Once You Know This, Building RAG Agents Becomes Easy

By: [Nate Herk]

The core problem most people run into

One of the most common questions I get is some version of:

"My agent is not answering correctly. How do I fix it?"

The answer is almost never "use a better model" or "tweak the prompt." The real issue is usually that the agent is looking at the wrong data, or not enough of it.

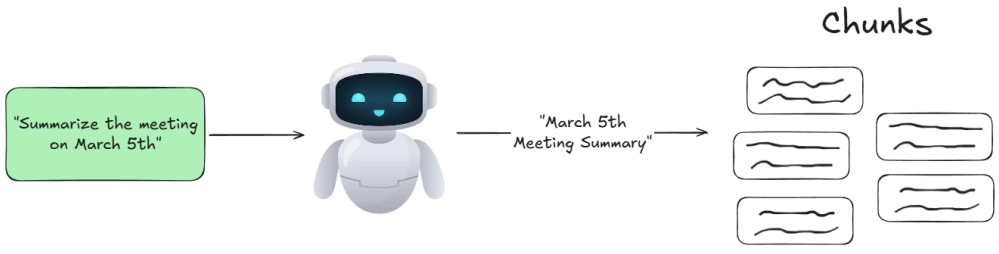
Before you choose any RAG setup, you need to be clear on two things:

- What kinds of questions will this agent be asked?
- What information does it need to see to answer accurately?

Once you understand that, choosing the right retrieval method becomes straightforward.

The problem with chunk based retrieval

Chunk based retrieval is when large documents are split into smaller pieces, embedded, and stored in a vector database. When a user asks a question, only the most relevant chunks are retrieved.



This works well for search style questions, but it breaks down when full context is required.

Why chunking can cause incorrect answers

- The agent loses the overall structure of the document.
- Retrieved chunks may come from different sources or timeframes.
- Summaries are based only on retrieved chunks, not the full dataset.

Example: long documents

If you embed a 20 page PDF and ask for a summary, the agent will summarize only the chunks it retrieves, not the full document. That means important sections may be skipped entirely.

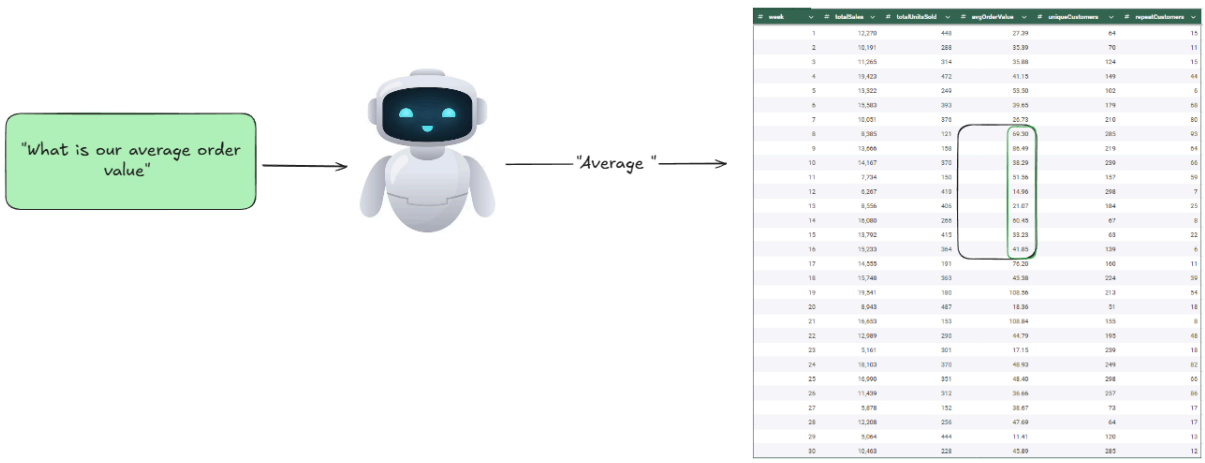
Example: tabular data

Chunking is especially dangerous with tables.

If you ask:

- "What week had the highest sales?"

The agent may only see a small slice of rows and pick the highest value *within that slice*, not across the full dataset.



The same problem happens with averages, totals, and rankings. The agent calculates using partial data and gives a confident but wrong answer.

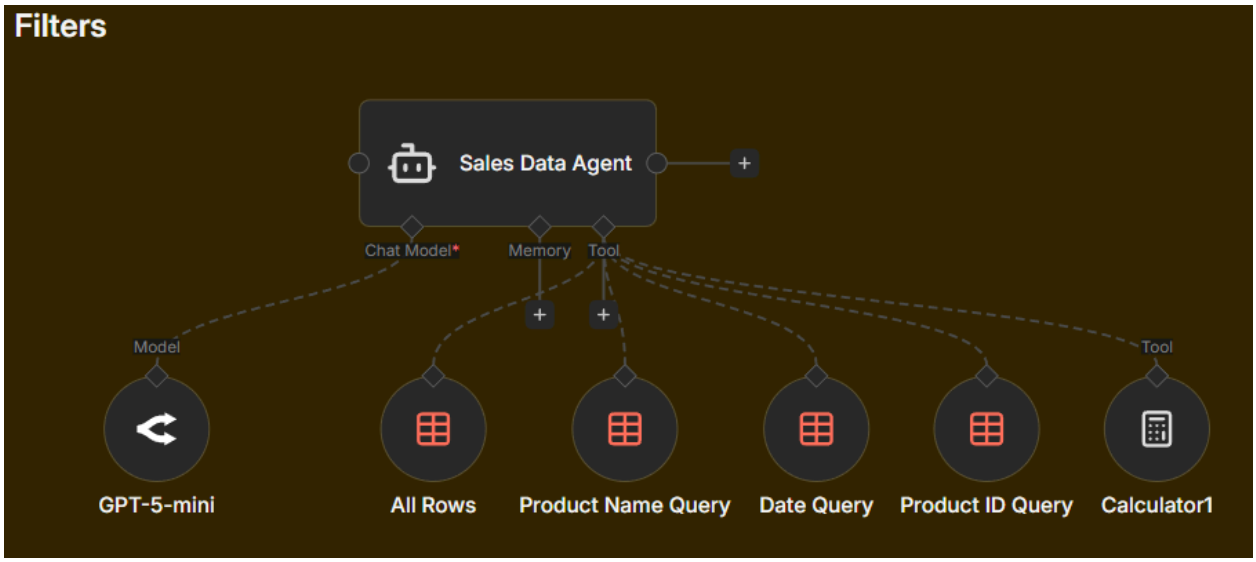
This is not a model problem. It is a retrieval problem.

The four retrieval methods covered in this video

Below are the four main ways you can give an agent access to data, ordered from simplest to most advanced.

---

1. Simple database filters



What this is

You are telling the system to only return rows that match specific rules.

Examples of rules:

- Product equals X
- Date equals Y
- Status equals open

When to use it

- Your data is structured in rows and columns.
- You know exactly which fields matter.
- The question can be answered with a small subset of records.

Common examples

- How many orders did we have today?
- Show me all open support tickets.
- Total revenue for product A this week.

Why it works well

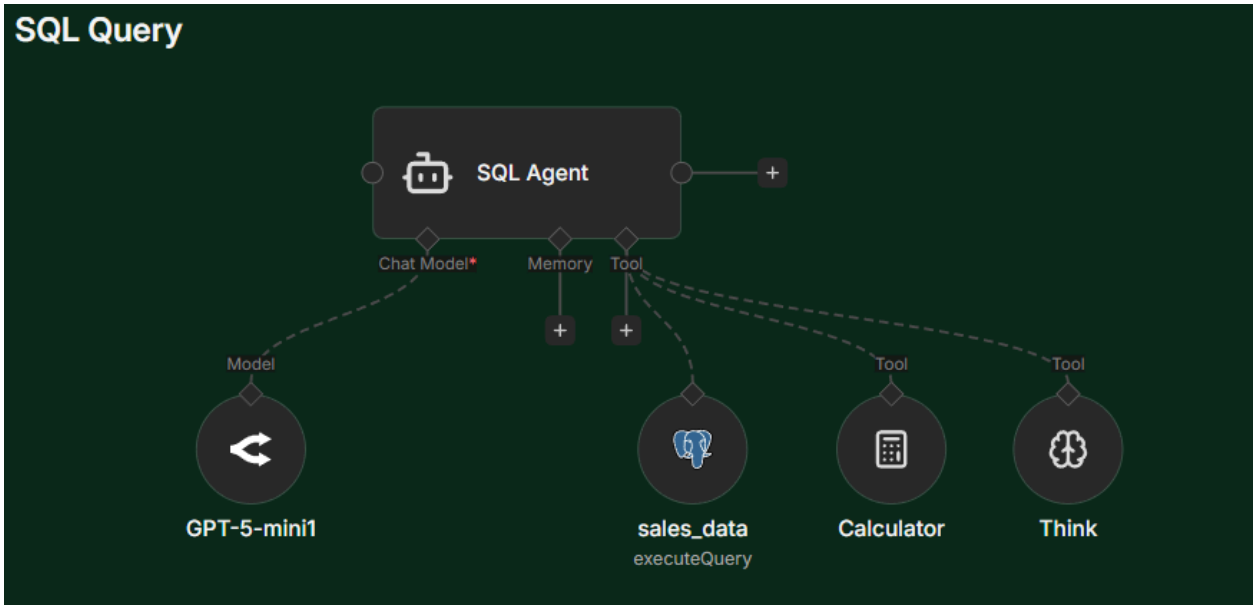
- Fast
- Cheap
- Very accurate
- Scales well

Beginner rule of thumb

If a human would use filters in a spreadsheet, use filters in n8n.

---

2. SQL queries



What this is

The agent generates a query that performs grouping, sorting, and calculations directly in the database.

When to use it

- You need totals, averages, or rankings.

- The question involves many rows.
- You need to compare or combine data.

Common examples

- Top 5 products by revenue
- Average order value by month
- Customers with the highest lifetime spend

Why it works well

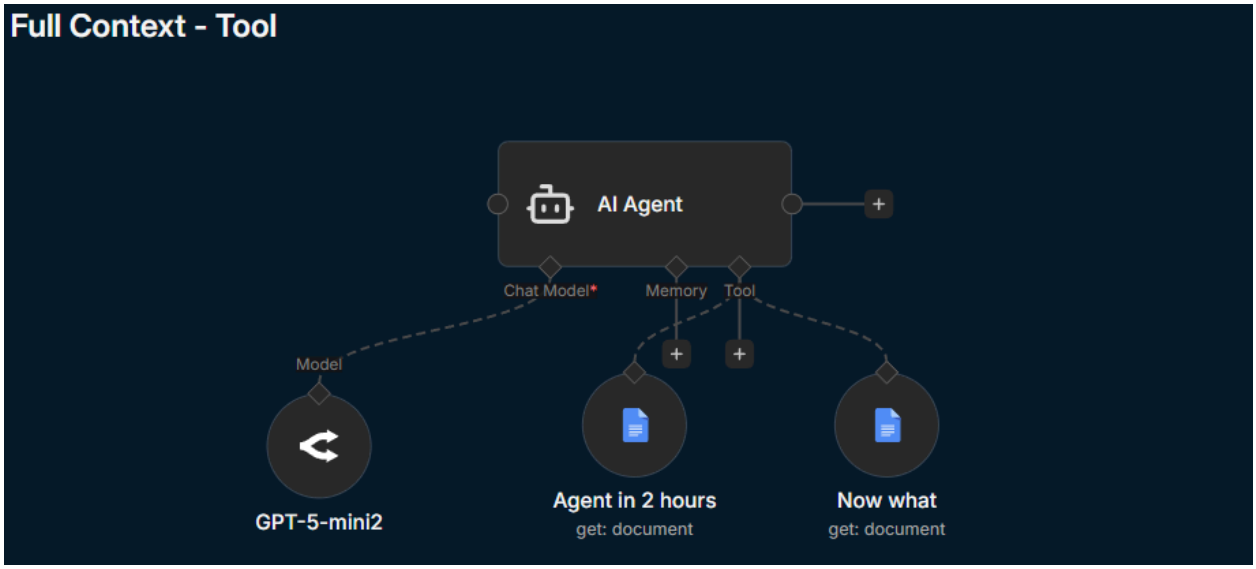
- Databases are built for this type of work.
- More reliable than having the AI reason over raw rows.
- Cheaper and more accurate than vector search for structured data.

Beginner rule of thumb

If a human would use pivot tables or formulas, use SQL.

---

3. Full context retrieval



What this is

The agent reads the entire document or set of documents at once. Nothing is chunked and order is preserved.

When to use it

- You need summaries or timelines.
- Order matters.
- The dataset fits in the model context window.

Common examples

- Summarize this PDF from start to finish
- Explain the key ideas in this training guide
- List the steps in this process

Why it works well

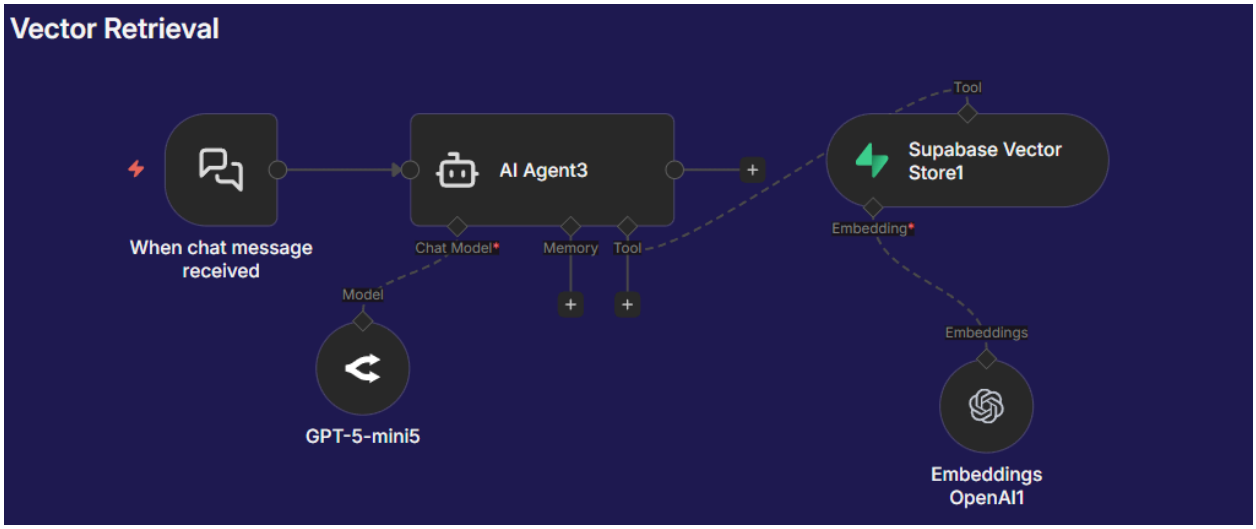
- Highest accuracy for long form content
- No missing context
- No mixing information between sources

Beginner rule of thumb

If a human would read the whole document, use full context.

---

4. Chunk based retrieval (vector search)



What this is

Documents are split into chunks and stored as embeddings. The agent retrieves only the most relevant pieces for a question.

When to use it

- You have a large knowledge base.
- Users ask open ended or search style questions.
- Full document order is not required.

Common examples

- What does our refund policy say?
- How does authentication work?
- What integrations do we support?

Why it works well

- Scales to large datasets
- Faster and cheaper than full context
- Great for semantic question answering

Where beginners get tripped up

- Incomplete summaries
- Wrong timelines
- Confident but incorrect answers

Beginner rule of thumb

If users are asking search style questions, use chunk based retrieval.

Final takeaway

Before you reach for a vector database, ask yourself:

- Does the agent need full context?
- Does this involve math or aggregation?
- Would a human filter, calculate, or read?

Answer those correctly, and building reliable RAG agents becomes much easier.
